Variable Aggregation Documentation

Code

Step 2: NLSY79 data aggregation pipeline

Katrine Wøldike and Regitze Holt

December 17, 2025

Note: This documentation was created with assistance from Claude 4.5.

Executive Summary

This document describes the dual-branch aggregation and machine learning preparation pipeline that transforms classified NLSY79 variables into two ML-ready datasets with different aggregation strategies.

Input: `nlsy79_classified.parquet` (7,016 × 11,981)
Outputs: - Highly Aggregated: `step_2_highly_aggregated.parquet` (6,790 × 3,303) - Mildly Aggregated: `step_2_mildly_aggregated.parquet` (6,790 × 5,942)

Note: 226 participants with >25% missing data are removed at the start of Step 2.

Manual Corrections Applied: - 648 type corrections from review file - 155 subgroup assignments (YES1/YES2 splits) - 68 variables excluded from aggregation

Pipeline Results:

Branch 1: Highly Aggregated (≥3 timepoints) - Variable groups identified: 675 groups from 9,392 variables - Aggregated features created: 1,548 summary features - Non-aggregated variables: 2,589 kept as-is (demographics, static features, <3 timepoints) - Combined dataset: 4,137 features before filtering - Missing values before imputation: 1,307,159 (4.65% of data) - Low-variance filtering: Removed 834 variables (≥95% identical) - Final dataset: 3,303 features (72.4% dimensionality reduction) - Participants retained: 6,790 (226 removed due to >25% missingness)

Branch 2: Mildly Aggregated (≥30 timepoints) - Variable groups identified: 3 groups from 3,894 variables - Aggregated features created: 8 summary features - Non-aggregated variables: 8,087 kept as-is (most variables, <30 timepoints) - Combined dataset: 8,095 features before filtering - Missing values before imputation: 4,234,813 (7.70% of data) - Low-variance filtering: Removed 2,153 variables (≥95% identical) - Final dataset: 5,942 features (50.4% dimensionality reduction) - Participants retained: 6,790 (same as Branch 1)

Key Innovation: Dual-branch approach provides both aggressive aggregation (maximizing dimensionality reduction) and conservative aggregation (preserving more raw temporal data), enabling comparison of aggregation strategies in downstream ML tasks.

Overview

Purpose

The Step 2 pipeline addresses three objectives:

1. Aggregation: Convert longitudinal series (3+ timepoints) into summary features
2. Feature Integration: Combine aggregated features with non-temporal variables
3. ML Preparation: Clean, impute, filter, and scale for machine learning

Why Aggregation?

Problem: 11,979 raw variables include extensive temporal redundancy

Example:

```
INCOME_1979, INCOME_1980, ..., INCOME_2022  (44 variables)
```

Solution: Aggregate into trajectory features

```
INCOME_mean, INCOME_trend, INCOME_std  (3 variables)
```

Benefits: - Captures temporal dynamics (trends, volatility, shifts) - Reduces dimensionality (44 → 3 variables) - Enables longitudinal pattern detection - Removes noise from single timepoint measurements

---

Pipeline Architecture

The Step 2 pipeline consists of shared preprocessing followed by two parallel aggregation branches:

```
1. Load Classified Data (11,981 variables)
   + Optional: Load Manual Corrections (inconsistent_variables_review.csv)
   ↓
2. Filter Participants (remove >25% missing) (6,790 participants)
   ↓
3. Group Variables by Base Name (3,009 groups)
   + Apply Subgroup Assignments (YES1→g1, YES2→g2)
   + Exclude Variables (aggregate_with_group=NO)
   ↓
   ┌──────────────────────────────┬──────────────────────────────┐
   │  BRANCH 1: HIGHLY AGGREGATED │  BRANCH 2: MILDLY AGGREGATED │
   │  (≥3 timepoints)             │  (≥30 timepoints)            │
   ├──────────────────────────────┼──────────────────────────────┤
   │ 4a. Filter Groups (675 groups)  │ 4b. Filter Groups (3 groups)  │
   │ 5a. Aggregate (1,548 features)  │ 5b. Aggregate (8 features)    │
   │ 6a. Combine (4,137 total)       │ 6b. Combine (8,095 total)     │
   │ 7a. Reclassify Variables        │ 7b. Reclassify Variables      │
   │ 8a. Impute Missing Values       │ 8b. Impute Missing Values     │
   │ 9a. Filter Low Variance         │ 9b. Filter Low Variance       │
   │ 10a. Save (3,303 features)      │ 10b. Save (5,942 features)    │
   └──────────────────────────────┴──────────────────────────────┘

Output Files:
- step_2_highly_aggregated.parquet / step_2_highly_classifications.csv
- step_2_mildly_aggregated.parquet / step_2_mildly_classifications.csv
- aggregated_groups_highly.csv / aggregated_groups_mildly.csv
- variance_report_highly.csv / variance_report_mildly.csv
- pipeline_summary_highly.csv / pipeline_summary_mildly.csv
```

Stage 1: Load Classified Data

Input File: `nlsy79_classified.parquet` (from Step 1)

Properties: - Dimensions: 7,016 participants × 11,981 variables (before participant filtering) - Variable types: Binary, Categorical (Ordinal/Nominal), Continuous - Missing values: Standardized to `np.nan` - Variable names: Descriptive (renamed in Step 1) - Month variables: Classified as categorical_nominal (cyclical data)

Pre-Processing from Step 1:

Outlier Handling: A conservative outlier filter was applied in Step 1 to numeric variables using the 3×IQR rule: - Detection threshold: Values beyond Q1 - 3×IQR or Q3 + 3×IQR flagged as potential outliers - 2% identical value rule: If 2% or more of detected outliers in a variable share the same value, all outliers for that variable are skipped (indicates valid extreme measurements, not data errors) - Proximity protection: Any flagged

outlier closer to the mean than the most extreme frequent value is also skipped - Result: Converted 14,734 outlier values to `np.nan` in 50 variables - Preservation: Frequent extreme values retained as valid measurements - Rationale: Balances outlier removal with preservation of valid extreme measurements in skewed distributions

Participant Filtering (Step 2.1a): - Removes participants with >25% missing data across all variables - Removed: 226 participants - Retained: 6,790 participants - Rationale: High missingness participants provide limited information for ML models

Classifications File: `variable_classifications.csv` - Maps each variable to its type - Used for type-specific aggregation strategies

Hide

```
df = pd.read_parquet('nlsy79_classified.parquet')
classifications = pd.read_csv('variable_classifications.csv')
variable_types = dict(zip(classifications['variable_name'],
                          classifications['variable_type']))
```

Manual Corrections (Optional)

Input File: `inconsistent_variables_review.csv`

Step 1 identifies 58 variable groups with inconsistent types across timepoints (e.g., `INCOME_1990` classified as continuous but `INCOME_2000` as ordinal). These groups require manual review to determine correct handling.

Review CSV Structure:

Column	Description	Values
`group_id`	Unique group identifier	1-58
`base_name`	Variable base name	e.g., “INCOME”
`variable_name`	Full variable name	e.g., “INCOME_1990”
`current_type`	Step 1 classification	binary, ordinal, etc.
`assigned_type`	Most common type in group	(for reference)
`corrected_type`	Manual input: corrected type	User fills in
`aggregate_with_group`	Manual input: aggregation decision	YES / NO / YES1 / YES2
`notes`	Manual input: reasoning	User comments

Three Manual Correction Methods:

1. Type Correction

Fill `corrected_type` column to override original classification:

Hide

```
# Example: Correct misclassified variable
row: INCOME_2000, current_type=ordinal, corrected_type=continuous
```

Step 2 uses corrected type for aggregation strategy.

2. Subgroup Splitting

Use `YES1`, `YES2`, `YES3` in `aggregate_with_group` column to split group into separate aggregates:

Hide

```
# Example: Job satisfaction scale changed midway through survey
SATISFACTION_1979: aggregate_with_group = YES1  → satisfaction_g1_mean
SATISFACTION_1980: aggregate_with_group = YES1  → satisfaction_g1_std
SATISFACTION_2000: aggregate_with_group = YES2  → satisfaction_g2_mean
SATISFACTION_2010: aggregate_with_group = YES2  → satisfaction_g2_std
```

Subgroups are named `{base_name}_g1`, `{base_name}_g2`, etc.

Use case: Survey instrument changed, making temporal aggregation inappropriate across all timepoints.

3. Exclude from Aggregation

Set `aggregate_with_group = NO` to keep variable as individual non-aggregated feature:

Hide

```
# Example: Variable conceptually doesn't belong in temporal aggregation
BASELINE_ASSESSMENT_1979: aggregate_with_group = NO
```

Variable retained as single feature, not aggregated.

Use case: Measurement is not truly longitudinal or shouldn't be summarized.

Loading Manual Corrections

Hide

```
REVIEW_FILE = "inconsistent_variables_review.csv"

if os.path.exists(REVIEW_FILE):
    review_df = pd.read_csv(REVIEW_FILE)

    # Apply type corrections
    for _, row in review_df.iterrows():
        var_name = row['variable_name']
        corrected_type = row.get('corrected_type', '')

        if pd.notna(corrected_type) and corrected_type.strip() != '':
            variable_types[var_name] = corrected_type.strip()

    # Handle subgroup assignments (YES1 → g1, YES2 → g2)
    for _, row in review_df.iterrows():
        aggregate_flag = row.get('aggregate_with_group', 'YES')
        if aggregate_flag not in ['YES', 'NO', '']:
            base_name = row['base_name']
            group_suffix = aggregate_flag.replace('YES', 'g')
            subgroup_assignments[var_name] = f"{base_name}_{group_suffix}"

    # Identify variables to exclude (NO)
    no_aggregate_vars = review_df[
        review_df['aggregate_with_group'] == 'NO'
    ]['variable_name'].tolist()
```

Console Output:

```
Loading manual corrections from inconsistent_variables_review.csv...
  Applied 648 type corrections
  Found 155 subgroup assignments (YES1/YES2)
Loaded: 6790 rows × 11981 columns
Classifications: 11981 variables

Grouping longitudinal variables by base name...
  Excluding 68 variables marked with aggregate_with_group=NO
  Found 2948 variable groups
  Retained 665 groups with ≥3 timepoints
```

Stage 2: Group Variables by Base Name

Objective

Identify longitudinal series by extracting base names from temporal variables.

Base Name Extraction

Pattern Recognition: Remove common temporal suffixes to extract base variable names.

Hardcoded Aggregations: - STATUS* → All `STATUS` variables aggregate together (regardless of full name) - HRS_WO* → All `HRS_WO` variables aggregate together (regardless of full name)

Suffixes Removed: - Year suffixes: `_1979`, `_1980`, …, `_2022` - Numeric identifiers: `.01`, `.02`, `.03`, … - Cross-round indicators: `_XRND`, `_TNEW_XRND`

Example Transformations:

```
INCOME_WAGES_SALARY_1979 → INCOME_WAGES_SALARY
INCOME_WAGES_SALARY_2000 → INCOME_WAGES_SALARY
INCOME_WAGES_SALARY_2022 → INCOME_WAGES_SALARY

EMPLOYERS_STARTDATE.01_XRND → EMPLOYERS_STARTDATE
EMPLOYERS_STARTDATE.02_XRND → EMPLOYERS_STARTDATE
EMPLOYERS_STARTDATE.03_XRND → EMPLOYERS_STARTDATE
```

Implementation:

Hide

```
def extract_base_name(var_name):
    patterns = [
        r'_19\d{2}$', r'_20\d{2}$',  # Year suffixes
        r'\.\d{1,2}$',               # Numeric identifiers
        r'_XRND$', r'_TNEW_XRND$'    # Cross-round indicators
    ]

    base = var_name
    for pattern in patterns:
        base = re.sub(pattern, '', base)

    return base if base != var_name else None
```

Grouping Results

Total Groups Identified: 3,009

Examples: - `INCOME_WAGES_SALARY` : 44 timepoints (1979-2022) - `EMPLOYERS_STARTDATE` : 8 timepoints - `HEALTH_STATUS` : 26 timepoints (biennial 1979-2014) - `MARITAL_STATUS` : 26 timepoints - `STATUS` (hardcoded): All STATUS* variables aggregated together - `HRS_WO` (hardcoded): All HRS_WO* variables aggregated together

Subgroup Assignments:

Variables with `YES1` / `YES2` in manual review CSV are split into separate groups: - `SATISFACTION_1979` (YES1) → grouped as `SATISFACTION_g1` - `SATISFACTION_2000` (YES2) → grouped as `SATISFACTION_g2`

Excluded Variables:

Variables marked `aggregate_with_group=NO` are excluded from grouping and kept as non-aggregated features.

Non-Groupable Variables: Variables without temporal patterns (demographics, IDs, single measurements) are not grouped and remain as individual features.

Variable Classification Distribution (Before Aggregation):

Type	Count	Percentage
Binary	3,967	33.1%
Continuous	4,554	38.0%
Categorical Nominal	2,626	21.9%
Categorical Ordinal	834	7.0%
Total	11,981	100%

Variables by Aggregation Status: - Variables to aggregate (in any branch): 11,868 - Variables to keep static (no temporal pattern): 113

Stage 3: Dual-Branch Filtering

After grouping variables by base name, the pipeline forks into two parallel branches with different timepoint thresholds:

Branch 1: Highly Aggregated (≥3 timepoints)

Threshold: Groups with 3 or more timepoints (see Key Design Decisions for rationale)

Results: - Groups retained: 675 - Variables aggregated: 9,392 → 1,548 features - Reduction: 84% within aggregated variables

Branch 2: Mildly Aggregated (≥30 timepoints)

Threshold: Groups with 30 or more timepoints

Results: - Groups retained: 3 - Variables aggregated: 3,894 → 8 features - Reduction: 99.8% within aggregated variables (but only 32.5% of total variables)

Comparison: Highly vs. Mildly Aggregated

Metric	Highly (≥3)	Mildly (≥30)
Groups aggregated	675	3
Variables aggregated	9,392	3,894
Aggregated features created	1,548	8
Non-aggregated kept	2,589	8,087

Total features (before filtering)	4,137	8,095
Final features (after filtering)	3,303	5,942
Dimensionality reduction	72.4%	50.4%

Key Insight: The highly aggregated branch maximizes feature reduction, while the mildly aggregated branch preserves more granular temporal information. Both branches undergo identical processing after aggregation (imputation, variance filtering, etc.).

Stage 4: Type-Specific Aggregation

Note: This stage runs independently in both branches (highly and mildly) using their respective filtered groups.

Aggregation Strategy

Variables aggregated using type-specific functions based on Step 1 classifications.

Type-Specific Approaches:

Type	Aggregation Functions
Binary	Proportion positive, proportion shifts
Ordinal	Mean, standard deviation, linear trend
Nominal	Mode, Shannon entropy
Continuous	Mean, standard deviation, linear trend

Binary Aggregation

Input: Binary time series (0/1 values)

Features Created: 1. Proportion Positive: `mean(values)` - Interpretation: % of timepoints with value=1 - Range: [0, 1]

2. Proportion Shifts: `count(changes) / (n-1)`
   - Interpretation: Temporal stability (low) vs volatility (high)
   - Range: [0, 1]

Example:

```
EMPLOYED: [1, 1, 0, 1, 1, 0, 1] (7 timepoints)

→ employed_prop_positive = 5/7 = 0.714  (employed 71.4% of time)
→ employed_prop_shifts = 4/6 = 0.667    (high turnover)
```

Implementation:

Hide

```python
def aggregate_binary(df, cols, base_name):
    subset = df[cols].copy()
    n_vars = len(cols)

    # Proportion positive
    prop_positive = subset.mean(axis=1)

    # Proportion of shifts (changes)
    shifts = (subset.diff(axis=1) != 0).sum(axis=1)
    prop_shifts = shifts / (n_vars - 1)

    return pd.DataFrame({
        f"{base_name}_prop_positive": prop_positive,
        f"{base_name}_prop_shifts": prop_shifts
    })
```

Ordinal Aggregation

Input: Ordered categorical time series (Likert scales, education levels)

Features Created: 1. Mean Trajectory: `mean(values)` - Central tendency over time

2. Standard Deviation: `std(values)`
   - Temporal variability
3. Linear Trend: `slope` from linear regression
   - Direction and rate of change

- Positive: increasing over time
- Negative: decreasing over time

Example:

```
HEALTH_STATUS: [1, 2, 2, 3, 3, 4] (1=Excellent ... 5=Poor)

→ health_mean = 2.5         (average health "good")
→ health_std = 1.05         (moderate variation)
→ health_trend = +0.6       (declining health over time)
```

Implementation:

Hide

```python
def aggregate_ordinal(df, cols, base_name):
    subset = df[cols].copy()

    mean_vals = subset.mean(axis=1)
    std_vals = subset.std(axis=1)

    # Linear trend (time-weighted)
    def compute_trend(row):
        vals = row.dropna()
        if len(vals) < 2:
            return 0
        x = np.arange(len(vals))
        slope, _ = np.polyfit(x, vals.values, 1)
        return slope

    trends = subset.apply(compute_trend, axis=1)

    return pd.DataFrame({
        f"{base_name}_mean": mean_vals,
        f"{base_name}_std": std_vals,
        f"{base_name}_trend": trends
    })
```

Nominal Aggregation

Input: Unordered categorical time series (occupation, race, cyclical months)

Features Created: 1. Mode: Most frequent category - Dominant category over time

2. Shannon Entropy: `-Σ(p_i × log2(p_i))`
    - Categorical diversity measure
    - Low entropy: Stable (same category)
    - High entropy: Variable (many different categories)

Example:

```
OCCUPATION: [2, 2, 5, 2, 2, 2] (6 timepoints)
Categories: 1=Professional, 2=Manager, 3=Sales, 4=Clerical, 5=Technical

→ occupation_mode = 2           (Manager most frequent)
→ occupation_entropy = 0.92     (some variation, mostly stable)
```

Entropy Interpretation: - 0 bits: No variation (always same category) - 1 bit: Evenly split between 2 categories - 2 bits: Evenly split among 4 categories - 3+ bits: High diversity across many categories

Implementation:

Hide

```
def aggregate_nominal(df, cols, base_name):
    subset = df[cols].copy()

    # Mode (most frequent category)
    mode_vals = subset.mode(axis=1).iloc[:, 0]

    # Shannon entropy (categorical diversity)
    def compute_entropy(row):
        vals = row.dropna()
        if len(vals) == 0:
            return 0
        value_counts = vals.value_counts()
        probs = value_counts / len(vals)
        return -np.sum(probs * np.log2(probs + 1e-10))

    entropy_vals = subset.apply(compute_entropy, axis=1)

    return pd.DataFrame({
        f"{base_name}_mode": mode_vals,
        f"{base_name}_entropy": entropy_vals
    })
```

Continuous Aggregation

Input: Numeric measurement time series (income, hours worked, weight)

Features Created: 1. Mean: `mean(values)` - Average level across time

2. Standard Deviation: `std(values)`
    - Temporal volatility
3. Linear Trend: `slope` from regression
    - Rate of change over time

Example:

```
INCOME: [25000, 28000, 32000, 35000, 38000, 42000]

→ income_mean = $33,333      (average income)
→ income_std = $6,055        (moderate volatility)
→ income_trend = +$3,400/yr  (strong growth)
```

Implementation:

Hide

```
def aggregate_continuous(df, cols, base_name):
    subset = df[cols].copy()

    mean_vals = subset.mean(axis=1)
    std_vals = subset.std(axis=1)

    # Linear trend
    def compute_trend(row):
        vals = row.dropna()
        if len(vals) < 2:
            return 0
        x = np.arange(len(vals))
        slope, _ = np.polyfit(x, vals.values, 1)
        return slope

    trends = subset.apply(compute_trend, axis=1)

    return pd.DataFrame({
        f"{base_name}_mean": mean_vals,
        f"{base_name}_std": std_vals,
        f"{base_name}_trend": trends
    })
```

Aggregation Results Summary

Feature Expansion by Type: - Binary groups → 2 features each (proportion, shifts) - Ordinal groups → 3 features each (mean, std, trend) - Nominal groups → 2 features each (mode, entropy) - Continuous groups → 3 features each (mean, std, trend)

Example Variable Names:

```
income_wages_salary_mean_n0044_s43
income_wages_salary_std_n0044_s43
income_wages_salary_trend_n0044_s43
health_status_mean_n0026_s35
health_status_std_n0026_s35
health_status_trend_n0026_s35
employed_prop_positive_n0026_s35
employed_prop_shifts_n0026_s35
```

Metadata Suffixes: Understanding Feature Names

Naming Convention: `{base_name}_{metric}_nXXXX_sYY`

Components: - base_name: Variable base name (e.g., `income_wages_salary`) - metric: Aggregation function (e.g., `mean` , `trend` , `prop_positive`) - nXXXX: Number of original timepoints aggregated (zero-padded, e.g., `n0044` = 44 variables) - sYY: Temporal span in years between first and last measurement (e.g., `s43` = 43 years)

Why Include Metadata?

Quality Assessment: - Features from many timepoints (n0100+) are more reliable - Features from few timepoints (n0003-n0005) may be noisy

Temporal Context: - Short spans (s01-s05) capture recent changes - Long spans (s30-s45) capture lifecycle trajectories

Example Interpretation:

```
income_wages_salary_mean_n0044_s43
|                    |    |     |
|                    |    |     └─ 43-year timespan (1979-2022)
|                    |    └──────── 44 timepoints aggregated
|                    └──────────── Mean trajectory
└──────────────────────────────── Variable: income wages/salary

Interpretation: Average income computed from 44 annual measurements
                spanning 43 years (comprehensive lifecycle earnings)
```

```
education_highest_degree_mean_n0003_s10
|                         |    |     |
|                         |    |     └─ 10-year timespan
|                         |    └─────── Only 3 timepoints
|                         └──────────── Mean value
└──────────────────────────────────── Variable: highest degree

Interpretation: Average education level from 3 measurements over
                10 years (sparse data, potentially less reliable)
```

Using Metadata for Feature Engineering:

Hide

```
# Filter features by timepoint count
high_quality = [col for col in df.columns if '_n' in col
                and int(col.split('_n')[1][:4]) >= 20]

# Filter features by temporal span
lifecycle_features = [col for col in df.columns if '_s' in col
                      and int(col.split('_s')[1][:2]) >= 30]
```

Stage 5: Combine with Non-Aggregated Variables

Note: This stage runs independently in both branches with different variable sets.

Rationale

Problem: Aggregation only captures temporal patterns

Missing Context: - Static demographics (gender, race, birth cohort) - Single-timepoint measurements - Variables with insufficient timepoints for aggregation (different per branch)

Solution: Combine aggregated features with non-aggregated variables

Non-Aggregated Variables (Branch-Specific)

Definition: Variables not included in aggregation for that branch

Branch 1 (Highly Aggregated - ≥3 timepoints): - Variables with <3 timepoints: kept as individual variables - Count: 2,589 non-aggregated variables

Branch 2 (Mildly Aggregated - ≥30 timepoints): - Variables with <30 timepoints: kept as individual variables - Count: 8,087 non-aggregated variables

Categories: 1. Static Demographics: Gender, race, birth year, sample flags 2. Single Timepoint: Baseline/final measurements, one-time modules 3. Insufficient Temporal Data: Below branch threshold

Combined Dataset (Per Branch)

Combination Strategy:

Hide

```
# Aggregated variables (source) → Dropped after aggregation
aggregated_vars = [var for group in groups for var in group]

# Non-aggregated variables → Kept as-is
non_aggregated_vars = [col for col in df.columns
                       if col not in aggregated_vars]

# Combine
df_combined = pd.concat([
    df[non_aggregated_vars],  # Non-aggregated
    df_aggregated             # Trajectory features
], axis=1)
```

Combined Dimensions:

Branch	Non-Aggregated	Aggregated	Total Combined
Highly (≥3)	2,589	1,548	4,137
Mildly (≥30)	8,087	8	8,095

Type Mapping: - Non-aggregated: Preserve original classifications from Step 1 - Aggregated: Reclassify based on summary statistic properties

---

Stage 6: Post-Aggregation Reclassification

Note: This stage runs independently in both branches.

Purpose

After aggregation, all variables in the combined dataset need classification: - Aggregated features: Newly created, need classification - Non-aggregated features: Preserve original Step 1 classifications

Reclassification Strategy

Hard-coded classifications based on feature names (deterministic and reliable):

Hide

```python
def classify_aggregated_variable(col):
    """
    Hard-coded classifications for aggregated features.

    Each aggregation function creates specific features with known types:
    - Binary: prop_positive, prop_shifts → continuous
    - Ordinal: mean, std, trend → continuous
    - Continuous: mean, std, trend → continuous
    - Nominal: mode → categorical_nominal, entropy → continuous
    - Year: mean, std, trend → continuous (year now aggregated as continuous)
    """
    # Nominal aggregation: mode stays categorical, entropy is continuous
    if '_mode_' in col:
        return 'categorical_nominal'
    if '_entropy_' in col:
        return 'continuous'

    # Binary aggregation: both proportions are continuous
    if '_prop_positive_' in col or '_prop_shifts_' in col:
        return 'continuous'

    # Ordinal/Continuous/Year aggregation: all are continuous
    if '_mean_' in col or '_std_' in col or '_trend_' in col:
        return 'continuous'

    # Default to continuous (should never reach here)
    return 'continuous'
```

Classification Results:

Original Variable Type	Aggregated Features	Classifications
Binary	`prop_positive` , `prop_shifts`	continuous, continuous
Ordinal	`mean` , `std` , `trend`	continuous, continuous, continuous
Continuous	`mean` , `std` , `trend`	continuous, continuous, continuous
Nominal	`mode` , `entropy`	categorical_nominal, continuous

Results: - Aggregated features: Classified by feature name patterns (deterministic) - Non-aggregated features: Original Step 1 classifications preserved - All variables now have assigned types for imputation

---

Stage 7: Missing Value Analysis & Type-Specific Imputation

Note: This stage runs independently in both branches and produces both imputed and non-imputed versions.

Stage 7a: Missing Value Analysis

Before Imputation: Comprehensive analysis of missing data patterns

Metrics Calculated: - Total missing values and percentage - Number of columns with missing values - Number of rows with missing values - Per-variable missing value counts and percentages

Output: `missing_values_report_{branch}.csv` - Lists all variables with their missing counts and percentages - Sorted by missing count (descending) - Used for transparency and quality assessment

Stage 7b: Imputation Strategy

Remaining Missing Values: After participant filtering, variables still have <25% missing data

Imputation Methods by Variable Type

Data Type	Strategy
Binary	Probabilistic sampling
Categorical Ordinal	Probabilistic sampling
Categorical Nominal	Probabilistic sampling
Continuous	Median

See Key Design Decisions section for detailed rationale on probabilistic sampling vs mode imputation and median vs mean imputation.

Probabilistic Imputation (Binary, Ordinal, Nominal):

Hide

```
def probabilistic_impute(series):
    """
    Impute missing values by sampling from observed distribution.
    Prevents inflation of the mode by preserving value frequencies.
    """
    # Get distribution of non-missing values
    value_counts = series.dropna().value_counts(normalize=True)

    # Sample from distribution for missing values
    missing_mask = series.isnull()
    n_missing = missing_mask.sum()

    if n_missing > 0:
        imputed_values = np.random.choice(
            value_counts.index,
            size=n_missing,
            p=value_counts.values
        )
        series.loc[missing_mask] = imputed_values

    return series

# Apply to categorical variables
for col in binary_cols + ordinal_cols + nominal_cols:
    df_imputed[col] = probabilistic_impute(df_combined[col])
```

Median Imputation (Continuous, Year):

Hide

```
from sklearn.impute import SimpleImputer

imputer_median = SimpleImputer(strategy='median')
df_imputed[continuous_cols] = imputer_median.fit_transform(df_combined[continuous_cols])
```

Imputation Results

Branch 1 (Highly Aggregated) - Variables Imputed by Type: - Binary: 855 variables (probabilistic sampling) - Ordinal: 308 variables (probabilistic sampling) - Nominal: 211 variables (probabilistic sampling) - Continuous: 2,763 variables (median imputation) - Total: 4,137 variables

Branch 2 (Mildly Aggregated) - Variables Imputed by Type: - Binary: 3,967 variables (probabilistic sampling) - Ordinal: 834 variables (probabilistic sampling) - Nominal: 695 variables (probabilistic sampling) - Continuous: 2,599 variables (median imputation) - Total: 8,095 variables

Output Versions: 1. Imputed: `step_2_{branch}_aggregated.parquet` (0 missing values) 2. Non-imputed: `step_2_{branch}_aggregated_notimputed.parquet` (original missing values preserved) - Highly Aggregated: 1,027,241 missing values - Mildly Aggregated: 3,296,531 missing values

Why Both Versions? - Imputed: Ready for models requiring complete data (e.g., neural networks, distance-based algorithms) - Non-imputed: For tree-based models (XGBoost, LightGBM) that handle missing values natively - Research flexibility: Compare model performance with/without imputation

Final Missing Values (Imputed Version): 0 (100% completeness)

Quality Check:

Hide

```
assert df_imputed.isnull().sum().sum() == 0
```

Stage 8: Low-Variance Filtering

Objective

Remove features with insufficient variation (≥95% identical values)

Rationale

Why Filter Low-Variance Features?

1. Minimal Predictive Power: Nearly constant features provide little information

2. Computational Cost: Slow training without benefit
3. Numerical Instability: Can cause issues in distance-based algorithms
4. Overfitting Risk: Models may spuriously fit noise in low-variance features

Variance Threshold

Threshold: 5% variance (remove if ≥95% identical)

Definition:

```
variance = 1 - (count of most common value) / (total participants)
```

Examples:

Low Variance (REMOVED):

```
Variable: EVER_MARRIED_1979
Distribution: 6,995 = Yes (99.1%), 65 = No (0.9%)
Variance: 0.9% → REMOVED
```

Sufficient Variance (KEPT):

```
Variable: INCOME_MEAN
Distribution: Continuous, well-distributed
Variance: High → KEPT
```

Variance Analysis

Process:

Hide

```
variance_data = []
for col in df_imputed.columns:
    vals = df_imputed[col].value_counts()
    most_common_pct = vals.iloc[0] / len(df_imputed) * 100
    variance = 100 - most_common_pct

    variance_data.append({
        'variable': col,
        'unique_values': len(vals),
        'most_common_pct': most_common_pct,
        'variance_pct': variance
    })
```

Filtering Results (Per Branch)

Branch 1: Highly Aggregated - Before filtering: 4,137 variables - Low variance removed (≥95%): 858 variables - After filtering: 3,279 variables - Retention rate: 79.3%

Branch 2: Mildly Aggregated - Before filtering: 8,095 variables - Low variance removed (≥95%): 2,218 variables - After filtering: 5,877 variables - Retention rate: 72.6%

Variance Reports: Saved to `variance_report_highly.csv` and `variance_report_mildly.csv`

Stage 9: Save Final Outputs

Output Files (Dual-Branch)

Each branch produces its own complete set of output files:

Branch 1: Highly Aggregated (≥3 timepoints)

1. step_2_highly_aggregated.parquet (Imputed Version)

Format: Apache Parquet (compressed)
Dimensions: 6,790 rows × 3,279 columns

Properties: - All missing values imputed using type-specific strategies (100% complete) - Low-variance features removed (≥95% threshold) - Unscaled (original units preserved) - Ready for models requiring complete data (neural networks, distance-based algorithms)

2. step_2_highly_aggregated_notimputed.parquet (Non-Imputed Version)

Format: Apache Parquet (compressed)
Dimensions: 6,790 rows × 3,279 columns (same features as imputed version)

Properties: - Missing values preserved from original data - Same variance filtering as imputed version - Unscaled (original units preserved) - Ready for tree-based models (XGBoost, LightGBM) that handle missing values natively - Enables comparison of imputation impact on model performance

3. step_2_highly_classifications.csv

Dimensions: 3,279 rows × 2 columns
Columns: `variable_name`, `variable_type`
Purpose: Type mapping for each feature (same for both imputed and non-imputed versions)

4. missing_values_report_highly.csv (NEW)

Content: Detailed missing value analysis before imputation
Columns: `variable`, `missing_count`, `missing_pct`
Purpose: Transparency on missing data patterns and imputation needs

5. aggregated_groups_highly.csv

Content: List of 675 aggregated groups
Columns: `base_name`, `n_timepoints`, `assigned_type`, `variables`
Purpose: Track which variables were aggregated together

6. variance_report_highly.csv

Content: Variance analysis for all variables pre-filtering
Columns: `variable`, `unique_values`, `most_common_pct`

7. pipeline_summary_highly.csv

Metrics: Complete pipeline statistics including missing value counts for this branch

8. inconsistent_variables_report_highly.txt

Content: Groups with mixed types (if any)

Branch 2: Mildly Aggregated (≥30 timepoints)

1. step_2_mildly_aggregated.parquet (Imputed Version)

Format: Apache Parquet (compressed)
Dimensions: 6,790 rows × 5,877 columns

Properties: - All missing values imputed using type-specific strategies (100% complete) - Low-variance features removed (≥95% threshold) - Unscaled (original units preserved) - More raw temporal data preserved (only very frequent measurements aggregated)

2. step_2_mildly_aggregated_notimputed.parquet (Non-Imputed Version)

Format: Apache Parquet (compressed)
Dimensions: 6,790 rows × 5,877 columns

Properties: - Missing values preserved from original data - Same variance filtering as imputed version - Unscaled (original units preserved) - Ready for tree-based models that handle missing values natively

3. step_2_mildly_classifications.csv

Dimensions: 5,942 rows × 2 columns
Columns: `variable_name`, `variable_type`
Purpose: Type mapping (same for both imputed and non-imputed versions)

4. missing_values_report_mildly.csv (NEW)

Content: Detailed missing value analysis before imputation
Columns: `variable`, `missing_count`, `missing_pct`

5. aggregated_groups_mildly.csv

Content: List of 3 aggregated groups
Columns: `base_name`, `n_timepoints`, `assigned_type`, `variables`

6. variance_report_mildly.csv

Content: Variance analysis for all variables pre-filtering

7. pipeline_summary_mildly.csv

Metrics: Complete pipeline statistics including missing value counts for this branch

8. inconsistent_variables_report_mildly.txt

Content: Groups with mixed types (if any)

Shared Input Files

**7. inconsistent_variables_review.csv

Format: CSV for manual review (8 columns, ~650 rows)

Purpose: Enable domain expert correction of hybrid variable classifications

Workflow: 1. Step 1 generates template with inconsistent groups 2. User fills `corrected_type`, `aggregate_with_group`, `notes` columns 3. Step 2 automatically applies corrections when file exists

Three Correction Methods: - Type correction: Fill `corrected_type` to override original classification - Subgroup splitting: Use `YES1` / `YES2` to create `{base}_g1`, `{base}_g2` aggregates - Exclusion: Set `NO` to keep as individual non-aggregated feature

---

Pipeline Summary

Dual-Branch Comparison

Branch 1: Highly Aggregated (≥3 timepoints)

Stage	Rows	Columns	Description
Input	7,016	11,981	Classified data from Step 1
Participant Filtering	6,790	11,981	Removed 226 participants with >25% missing
Manual Corrections	6,790	11,981	Applied 648 type corrections, 155 subgroups, excluded 68
Variable Grouping	6,790	3,009 groups	Identified longitudinal series
Timepoint Filtering	6,790	675 groups	Kept groups with ≥3 timepoints (9,392 variables)
Aggregation	6,790	1,548	Created trajectory features
Combination	6,790	4,137	Added 2,589 non-aggregated variables
Reclassification	6,790	4,137	All variables typed
Missing Value Analysis	6,790	4,137	1,307,159 missing (4.65%)
Imputation	6,790	4,137	Probabilistic (categorical) + median (continuous)
Variance Filtering	6,790	3,303	Removed 834 low-variance (≥95% identical)
Output (imputed)	6,790	3,303	72.4% reduction, 0 missing
Output (not imputed)	6,790	3,303	72.4% reduction, 1,027,241 missing

Branch 2: Mildly Aggregated (≥30 timepoints)

Stage	Rows	Columns	Description
Input	7,016	11,981	Classified data from Step 1
Participant Filtering	6,790	11,981	Removed 226 participants with >25% missing
Manual Corrections	6,790	11,981	Applied 648 type corrections, 155 subgroups, excluded 68
Variable Grouping	6,790	3,009 groups	Identified longitudinal series
Timepoint Filtering	6,790	3 groups	Kept groups with ≥30 timepoints (3,894 variables)
Aggregation	6,790	8	Created trajectory features
Combination	6,790	8,095	Added 8,087 non-aggregated variables
Reclassification	6,790	8,095	All variables typed
Missing Value Analysis	6,790	8,095	4,234,813 missing (7.70%)
Imputation	6,790	8,095	Probabilistic (categorical) + median (continuous)
Variance Filtering	6,790	5,942	Removed 2,153 low-variance (≥95% identical)
Output (imputed)	6,790	5,942	50.4% reduction, 0 missing
Output (not imputed)	6,790	5,942	50.4% reduction, 3,296,531 missing

Dimensionality Reduction Comparison

Metric	Highly (≥3)	Mildly (≥30)
Input variables	11,981	11,981

Input participants	7,016	7,016
After participant filter	6,790 × 11,981	6,790 × 11,981
Groups aggregated	675 groups (9,392 vars)	3 groups (3,894 vars)
Aggregated features	1,548	8
Non-aggregated kept	2,589	8,087
Combined dataset	6,790 × 4,137	6,790 × 8,095
After variance filter	6,790 × 3,303	6,790 × 5,942
Dimensionality reduction	72.4%	50.4%

Overall Comparison: - Highly aggregated: 11,981 → 3,303 variables (72.4% reduction) - Mildly aggregated: 11,981 → 5,942 variables (50.4% reduction) - Participants: 7,016 → 6,790 (226 removed, 3.2% reduction)

Information Retention

✓ Completeness: 100% (0 missing values in imputed versions)
✓ Temporal Dynamics: Captured via trajectory features (mean, std, trend, etc.)
✓ Distribution Preservation: Probabilistic imputation for categorical variables
✓ Flexibility: Both imputed and non-imputed versions available
✓ Static Context: Non-aggregated variables preserved
✓ Variance: All features have ≥5% variation
✓ Scaling: Deferred (original units preserved for flexibility)
✓ Sample Size: 99.6% participants retained

Key Design Decisions

This section explains the rationale behind critical design choices in the aggregation pipeline.

1. Why Dual-Branch Processing?

Design: Two parallel processing branches with different timepoint thresholds (≥3 for highly, ≥30 for mildly)

Rationale: - Research Flexibility: Enables comparison of aggressive vs conservative aggregation strategies - Information-Reduction Tradeoff: Highly aggregated maximizes dimensionality reduction; mildly aggregated preserves temporal granularity - Model Comparison: Different models may benefit from different aggregation levels (neural networks vs tree-based) - Temporal Density: Some patterns require high-frequency data (e.g., 30+ timepoints), while others emerge from sparse measurements (3-5 timepoints)

Alternative Considered: Single branch with one fixed threshold
Why Rejected: Would force choice between dimensionality reduction and information preservation; dual-branch provides both

2. Why Minimum 3 Timepoints for Highly Aggregated Branch?

Design: Groups with <3 timepoints excluded from highly aggregated branch

Rationale: - 2 timepoints: Insufficient for trend analysis (cannot fit linear regression with confidence) - 3+ timepoints: Enable trajectory characterization (mean level, temporal volatility, directional change)

Example:

```
2 timepoints: [25000, 30000]
→ Cannot determine if increasing trend or single fluctuation

3+ timepoints: [25000, 28000, 32000]
→ Clear increasing trajectory (trend = +3500/year)
```

Alternative Considered: Minimum 5 or 10 timepoints
Why Rejected: Would discard too many valuable longitudinal measurements

3. Why Type-Specific Aggregation?

Design: Different aggregation functions for each variable type

Rationale: Appropriateness of statistics depends on measurement scale

Type	Can Compute Mean?	Can Compute Trend?	Can Compute Mode?
Binary	✗ Nonsensical (“average yes/no”)	✓ Temporal trend	✓ Most frequent state
Ordinal	✓ Quasi-continuous	✓ Temporal trend	✓ Most frequent level

Nominal	✗ No inherent order	✗ No meaningful trend	✓ Most frequent category
Continuous	✓ Numeric average	✓ Temporal trend	✗ Mode less informative

Example:

```
Occupation codes (nominal): [1, 2, 2, 5, 2]
✓ Mode = 2 (most common occupation)
✓ Entropy = 1.52 (moderate diversity)
✗ Mean = 2.4 (meaningless - cannot "average" occupations)
✗ Trend = +0.8 (nonsensical - no inherent order)
```

Alternative Considered: One-size-fits-all aggregation (e.g., always compute mean)
Why Rejected: Produces statistically invalid features (averaging nominal categories)

4. Why Metadata Suffixes (_nXXXX_sYY)?

Design: Encode timepoint count and temporal span in variable names

Rationale: - Quality Assessment: Features from 100 timepoints more reliable than from 3 - Temporal Context: 40-year spans capture lifecycle trajectories; 5-year spans capture recent changes - Feature Engineering: Enables filtering by quality/span without external lookup - Reproducibility: Self-documenting feature provenance

Example:

```
income_mean_n0044_s43: High reliability (44 timepoints, 43-year lifecycle)
income_mean_n0003_s05: Lower reliability (3 timepoints, 5-year snippet)
```

Alternative Considered: Store metadata in separate CSV
Why Rejected: Requires cross-referencing; metadata not self-contained in feature names

5. Why Probabilistic Imputation for Categorical Variables?

Design: Sample missing categorical values from observed distribution

Traditional Approach (Mode Imputation):

```
Occupation: [1, 2, 2, NaN, NaN, 5]
→ Mode = 2
→ Imputed: [1, 2, 2, 2, 2, 5]  ← Artificially inflates frequency of "2"
```

Our Approach (Probabilistic Sampling):

```
Occupation: [1, 2, 2, NaN, NaN, 5]
→ Distribution: 1 (20%), 2 (40%), 5 (20%)
→ Sample missing with these probabilities
→ Imputed: [1, 2, 2, 1, 2, 5]  ← Preserves original distribution
```

Rationale: - Preserves frequency distribution - Prevents mode inflation (over-representation of most common category) - More realistic for machine learning (models see distribution, not artificial homogeneity)

Alternative Considered: Mode imputation
Why Rejected: Artificially reduces categorical diversity

6. Why Median Imputation for Continuous Variables?

Design: Impute missing continuous values with median (not mean)

Rationale: - Robustness: Median unaffected by outliers (mean is sensitive) - Skewed Distributions: Income, wealth, work hours are highly skewed - Preservation: Maintains distributional shape better than mean

Example:

```
Income: [20000, 25000, 30000, NaN, 500000]
Mean = 143,750 (inflated by outlier)
Median = 27,500 (representative of typical value)
```

Alternative Considered: Mean imputation
Why Rejected: Distorted by extreme values common in socioeconomic data

7. Why Variance Filter (≥95% Identical)?

Design: Remove variables where ≥95% of values are the same

Rationale: - Information Content: Near-constant variables provide minimal predictive signal - Dimensionality Reduction: Eliminates redundant features - Model Efficiency: Reduces computational cost without information loss - Stability: Prevents overfitting to noise in rare non-constant values

Example:

```
EVER_EMPLOYED: [1, 1, 1, 1, 1, 1, 1, 1, 1, 0] (90% identical)
→ Kept (10% variance = sufficient information)

PARTICIPATED_IN_RARE_PROGRAM: [0, 0, 0, 0, ..., 0, 1] (99% identical)
→ Removed (1% variance = insufficient information)
```

Alternative Considered: Keep all variables regardless of variance
Why Rejected: Clutters feature space with uninformative variables

8. Why Both Imputed and Non-Imputed Versions?

Design: Save two versions of each branch (imputed and non-imputed)

Rationale:

Imputed Version: - Required for: Neural networks, distance-based models (KNN, SVM) - Advantage: 0% missing values; full dataset usable - Trade-off: Introduces imputation uncertainty

Non-Imputed Version: - Required for: Tree-based models (XGBoost, Random Forest, LightGBM) - Advantage: Missing values convey information (e.g., “did not work in 1985” vs “unknown”) - Trade-off: Cannot use with distance-based algorithms

Example:

```
Participant A: income_1990 = NaN (unemployed)
Participant B: income_1990 = 50000 (employed)

Imputed version: A gets median income (hides unemployment signal)
Non-imputed version: A keeps NaN (tree models can split on "missing vs not missing")
```

Alternative Considered: Single imputed version only
Why Rejected: Loses information content of missingness for tree-based models

9. Why Preserve Original Units (No Scaling)?

Design: Variables retain original scales and units (scaling disabled)

Rationale: - Interpretability: Features remain in natural units (dollars, years, percentages) - Flexibility: Users can apply preferred scaling method for their models - Feature Engineering: Easier to create domain-specific transformations - Tree-Based Models: Do not require scaling (scale-invariant)

When to Apply Scaling (User Responsibility): - Neural networks: Require standardization (μ=0, σ=1) - Distance-based models (KNN, SVM): Require normalization - Gradient-based optimization: Benefits from scaling

Example:

Hide

```
# User applies scaling as needed for their model
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_continuous)
```

Alternative Considered: Pre-scale all continuous variables
Why Rejected: Reduces interpretability; may not match user’s scaling needs

---

Quality Assurance

Validation Checks

1. Missing Values Check

Test: All missing values imputed

Hide

```
assert df_ml.isnull().sum().sum() == 0
```

Result: ✓ Passed (0 missing values)

2. Scaling Check

Status: Scaling is currently disabled in the pipeline.

Original Test (when scaling enabled):

Hide

```
# Uncomment when scaling is re-enabled
# scaled_cols = [col for col in df_ml.columns
#                if combined_types[col] in ['continuous', 'categorical_ordinal', 'year']]
# assert np.allclose(df_ml[scaled_cols].mean(), 0, atol=1e-6)
# assert np.allclose(df_ml[scaled_cols].std(), 1, atol=1e-6)
```

Result: N/A (scaling disabled)

3. Variance Check

Test: All features have ≥5% variance

Hide

```
for col in df_ml.columns:
    variance = 1 - (df_ml[col].value_counts().iloc[0] / len(df_ml))
    assert variance >= 0.05
```

Result: ✓ Passed (all features have sufficient variance)

4. Dimensionality Check

Test: Final dataset has expected dimensions

Hide

```
assert df_ml.shape == (7060, 1424)
```

Result: ✓ Passed

5. Type Consistency Check

Test: Variable types match classifications

Hide

```
classifications = pd.read_csv('ml_ready_classifications.csv')
assert len(classifications) == df_ml.shape[1]
assert set(classifications['variable_name']) == set(df_ml.columns)
```

Result: ✓ Passed

Known Limitations

1. Trajectory Linearity Assumption

Issue: Linear trend assumes constant rate of change

Reality: Many trajectories are non-linear (exponential growth, plateaus, U-shapes)

Mitigation: - Consider polynomial trends for critical variables - Use piecewise linear models for segmented trends - Tree-based models handle non-linearity naturally

2. Missing Data Mechanism

Issue: Mode/median imputation assumes MAR (Missing At Random)

Risk: If missing is MNAR (Missing Not At Random), imputation introduces bias

Mitigation: - Sensitivity analysis with different imputation methods - Multiple imputation for uncertainty quantification - Consider missingness indicators as features

3. Temporal Spacing Ignored

Issue: Aggregation treats all timepoints as equally spaced

Reality: NLSY79 changed from annual (1979-1994) to biennial (1996-2022)

Mitigation: - Trend calculation accounts for timepoint positions - Consider time-weighted aggregations for critical analyses

4. Within-Person Correlation

Issue: Trajectory features may be highly correlated

Example: `income_mean` and `income_max` likely r>0.9

Mitigation: - VIF analysis to detect multicollinearity - PCA for dimensionality reduction if needed - Ridge/Lasso regression for regularization

---

Usage Examples

Load ML-Ready Dataset

Hide

```
import pandas as pd
import numpy as np

# Load data
df_ml = pd.read_parquet('NLSY79_aggregated.parquet')
classifications = pd.read_csv('ml_ready_classifications.csv')

print(f"Dataset shape: {df_ml.shape}")
print(f"Missing values: {df_ml.isnull().sum().sum()}")
print(f"Variable types:\n{classifications['variable_type'].value_counts()}")
```

Supervised Learning Example

Hide

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Assume target variable exists (not in NLSY79 by default)
# Example: predict high income (binary classification)
df_ml['high_income'] = (df_ml['income_wages_salary_mean'] > df_ml['income_wages_salary_mean'].median()).astype(int)

# Features and target
X = df_ml.drop(columns=['high_income'])
y = df_ml['high_income']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Train model (data already scaled!)
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))

# Feature importance
feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': model.feature_importances_
}).sort_values('importance', ascending=False)

print(feature_importance.head(20))
```

Unsupervised Learning Example

Hide

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# PCA for dimensionality reduction
pca = PCA(n_components=50, random_state=42)
X_pca = pca.fit_transform(df_ml)

print(f"Explained variance: {pca.explained_variance_ratio_.sum():.2%}")

# K-means clustering
kmeans = KMeans(n_clusters=5, random_state=42)
clusters = kmeans.fit_predict(X_pca)

# Visualize first 2 principal components
plt.figure(figsize=(10, 8))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters, cmap='viridis', alpha=0.5)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('K-Means Clustering (n=5) on PCA-Reduced Data')
plt.colorbar(label='Cluster')
plt.savefig('clustering_pca.png', dpi=300)
```

Inverse Transform Scaled Variables

Hide

```
# Load scaler parameters
scaler_params = pd.read_csv('scaler_parameters.csv')

# Inverse transform a specific variable
var_name = 'income_wages_salary_mean'
scaled_values = df_ml[var_name]

# Get original scale parameters
params = scaler_params[scaler_params['variable'] == var_name].iloc[0]
original_values = scaled_values * params['std'] + params['mean']

print(f"Scaled range: [{scaled_values.min():.2f}, {scaled_values.max():.2f}]")
print(f"Original range: [${original_values.min():,.0f}, ${original_values.max():,.0f}]")
```

Feature Selection

Hide

```
from sklearn.feature_selection import SelectKBest, f_classif

# Select top 100 features by ANOVA F-statistic
# (requires target variable y)
selector = SelectKBest(score_func=f_classif, k=100)
X_selected = selector.fit_transform(X, y)

# Get selected feature names
selected_features = X.columns[selector.get_support()]
print(f"Selected {len(selected_features)} features")
print(selected_features.tolist())

# Scores
feature_scores = pd.DataFrame({
    'feature': X.columns,
    'score': selector.scores_
}).sort_values('score', ascending=False)

print(feature_scores.head(20))
```

Next Steps: Model Development

Recommended Modeling Approaches

1. Tree-Based Models (Recommended)

Algorithms: - Random Forest - Gradient Boosting (XGBoost, LightGBM, CatBoost)

Advantages: - Handle mixed scales naturally (scaled + unscaled) - Robust to multicollinearity - Capture non-linear relationships - Automatic feature interaction detection

No Preprocessing Needed: Data already clean and complete

2. Linear Models

Algorithms: - Logistic Regression - Ridge/Lasso Regression - Support Vector Machines

Requirements: - Data already scaled ✓ - Check for multicollinearity (VIF analysis) - Consider regularization (Ridge/Lasso)

3. Neural Networks

Algorithms: - Multi-layer Perceptron (MLP) - Deep neural networks

Requirements: - Data already scaled ✓ - May benefit from additional normalization - Consider dropout for regularization

4. Dimensionality Reduction

Algorithms: - PCA (Principal Component Analysis) - t-SNE (visualization) - UMAP (visualization + clustering)

Use Cases: - Reduce 1,424 features to ~50-100 components - Visualization of participant similarity - Cluster analysis

Cross-Validation Strategy

Recommended: Stratified K-Fold (k=5 or k=10)

Hide

```
from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = []
for train_idx, val_idx in skf.split(X, y):
    X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
    y_train, y_val = y.iloc[train_idx], y.iloc[val_idx]

    model.fit(X_train, y_train)
    y_pred_proba = model.predict_proba(X_val)[:, 1]

    auc = roc_auc_score(y_val, y_pred_proba)
    scores.append(auc)

print(f"Cross-validation AUC: {np.mean(scores):.3f} ± {np.std(scores):.3f}")
```

Hyperparameter Tuning

Recommended: Bayesian Optimization or Grid Search

Hide

```
from sklearn.model_selection import RandomizedSearchCV

param_distributions = {
    'n_estimators': [100, 200, 500],
    'max_depth': [10, 20, 30, None],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4],
    'max_features': ['sqrt', 'log2', None]
}

random_search = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    param_distributions=param_distributions,
    n_iter=50,
    cv=5,
    scoring='roc_auc',
    n_jobs=-1,
    random_state=42
)

random_search.fit(X_train, y_train)
print(f"Best AUC: {random_search.best_score_:.3f}")
print(f"Best params: {random_search.best_params_}")
```

Reproducibility

System Requirements

Software: - Python 3.8+ - pandas 1.3+ - numpy 1.20+ - scikit-learn 0.24+ - scipy 1.5+ - pyarrow 6.0+ (Parquet support)

Hardware: - RAM: 16 GB minimum (32 GB recommended) - Storage: 5 GB for intermediate files - CPU: Multi-core (4+ cores recommended)

Running the Script

Command:

Hide

```
python step2_trajectory_and_ml.py
```

Expected Runtime: ~5-10 minutes

With Manual Corrections: 1. Run Step 1 to generate `inconsistent_variables_review.csv` 2. Fill in `corrected_type`, `aggregate_with_group`, `notes` columns 3. Rerun Step 2 - corrections automatically applied

Console Output:

```
================================================================================
STEP 2: aggregation & ML PREPARATION
================================================================================

[2.1] Loading classified data...
  Loading manual corrections from inconsistent_variables_review.csv...
    Applied 127 type corrections
    Found 38 subgroup assignments (YES1/YES2)
  Excluding 12 variables marked with aggregate_with_group=NO
  Loaded: 7085 rows × 11979 columns
  Classifications: 11979 variables

[2.2] Grouping longitudinal variables by base name...
  Found 6805 variable groups
  Retained 661 groups with ≥3 timepoints

[2.3] Defining aggregation functions...

[2.4] Aggregating longitudinal variables...
  Created 1526 aggregated features from 661 variable groups
  aggregated variables (to be dropped): 6600
  Non-aggregated variables (to keep): 10318

[2.5] Reclassifying all variables...
  Classified 11844 total variables (10318 non-aggregated + 1526 aggregated)

[2.6] Filtering by missingness...
  Removed 0 variables with >25% missingness
  Remaining variables: 11844
  Removed 25 participants with >25% missingness
  Remaining participants: 7060

[2.7] Imputing remaining missing values...
  Imputing binary/ordinal/nominal variables (mode)...
  Imputing continuous variables (median)...
  Imputing year variables (median)...
  Imputation complete. Missing values remaining: 0

[2.8] Filtering low-variance variables...
  Removed 10420 variables with ≥95% identical values
  Remaining variables: 1424

[2.9] Standardizing continuous/ordinal/year variables...
  Scaled 1423 variables

[2.10] Saving final ML-ready outputs...
  Saved: NLSY79_aggregated.parquet (7060 × 1424)
  Saved: ml_ready_classifications.csv (1424 variables)
  Saved: scaler_parameters.csv
  Saved: pipeline_summary.csv

================================================================================
TRAJECTORY & ML PREPARATION COMPLETE
================================================================================

Classified dataset: 7085 × 11979
  - aggregated variables (dropped): 6600
  - Non-aggregated variables (kept): 10318
After aggregation: 7085 × 1526
Combined (non-aggregated + aggregated): 7085 × 11844
After filtering: 7060 × 11844
ML-ready dataset: 7060 × 1424

Dimensionality reduction: 88.1%
Variables removed (low variance): 10420
Participants retained: 7060 / 7085 (99.6%)

✅ Data is ready for machine learning!
```

Validation & Verification

Quick Validation Script

Run this script to verify pipeline outputs:

Hide

```
import pandas as pd
import numpy as np

print("=" * 80)
print("NLSY79 STEP 2 VALIDATION")
print("=" * 80)

# Load highly aggregated datasets
df_highly_imp = pd.read_parquet('step_2_highly_aggregated.parquet')
df_highly_notimp = pd.read_parquet('step_2_highly_aggregated_notimputed.parquet')
print(f"\n✓ Highly Aggregated (Imputed): {df_highly_imp.shape}")
print(f"✓ Highly Aggregated (Not Imputed): {df_highly_notimp.shape}")
print(f"  Missing (imputed): {df_highly_imp.isnull().sum().sum()} (should be 0)")
print(f"  Missing (not imputed): {df_highly_notimp.isnull().sum().sum():,}")

# Load mildly aggregated datasets
df_mildly_imp = pd.read_parquet('step_2_mildly_aggregated.parquet')
df_mildly_notimp = pd.read_parquet('step_2_mildly_aggregated_notimputed.parquet')
print(f"\n✓ Mildly Aggregated (Imputed): {df_mildly_imp.shape}")
print(f"✓ Mildly Aggregated (Not Imputed): {df_mildly_notimp.shape}")
print(f"  Missing (imputed): {df_mildly_imp.isnull().sum().sum()} (should be 0)")
print(f"  Missing (not imputed): {df_mildly_notimp.isnull().sum().sum():,}")

# Check classifications
types_highly = pd.read_csv('step_2_highly_classifications.csv')
types_mildly = pd.read_csv('step_2_mildly_classifications.csv')
print(f"\n✓ Highly Classifications: {len(types_highly)} variables")
print(f"✓ Mildly Classifications: {len(types_mildly)} variables")

print("\nHighly Aggregated Type Distribution:")
print(types_highly['variable_type'].value_counts())

print("\nMildly Aggregated Type Distribution:")
print(types_mildly['variable_type'].value_counts())

# Check missing value reports
missing_highly = pd.read_csv('missing_values_report_highly.csv')
missing_mildly = pd.read_csv('missing_values_report_mildly.csv')
print(f"\n✓ Highly: {missing_highly['missing_pct'].mean():.2f}% avg missing per variable (before imputation)")
print(f"✓ Mildly: {missing_mildly['missing_pct'].mean():.2f}% avg missing per variable (before imputation)")

# Check variance reports
variance_highly = pd.read_csv('variance_report_highly.csv')
variance_mildly = pd.read_csv('variance_report_mildly.csv')
print(f"\n✓ Highly: {len(variance_highly)} variables evaluated for variance")
print(f"✓ Mildly: {len(variance_mildly)} variables evaluated for variance")

# Check aggregated groups
groups_highly = pd.read_csv('aggregated_groups_highly.csv')
groups_mildly = pd.read_csv('aggregated_groups_mildly.csv')
print(f"\n✓ Highly: {len(groups_highly)} variable groups aggregated")
print(f"✓ Mildly: {len(groups_mildly)} variable groups aggregated")

print("\n" + "=" * 80)
print("VALIDATION COMPLETE - All files loaded successfully!")
print("=" * 80)
```

Expected Output:

```
================================================================================
NLSY79 STEP 2 VALIDATION
================================================================================

✓ Highly Aggregated (Imputed): (6790, 3303)
✓ Highly Aggregated (Not Imputed): (6790, 3303)
  Missing (imputed): 0 (should be 0)
  Missing (not imputed): 1,027,241

✓ Mildly Aggregated (Imputed): (6790, 5942)
✓ Mildly Aggregated (Not Imputed): (6790, 5942)
  Missing (imputed): 0 (should be 0)
  Missing (not imputed): 3,296,531

✓ Highly Classifications: 3303 variables
✓ Mildly Classifications: 5942 variables

Highly Aggregated Type Distribution:
continuous              2763
binary                   855
categorical_nominal      211
categorical_ordinal      308
Name: variable_type, dtype: int64

Mildly Aggregated Type Distribution:
continuous              2599
binary                  3967
categorical_nominal      695
categorical_ordinal      834
Name: variable_type, dtype: int64

✓ Highly: 4.65% avg missing per variable (before imputation)
✓ Mildly: 7.70% avg missing per variable (before imputation)

✓ Highly: 4137 variables evaluated for variance
✓ Mildly: 8095 variables evaluated for variance

✓ Highly: 675 variable groups aggregated
✓ Mildly: 3 variable groups aggregated

================================================================================
VALIDATION COMPLETE - All files loaded successfully!
================================================================================
```

Verification Checklist

1. Check output files exist:

Hide

```
# Highly aggregated branch
ls step_2_highly_aggregated.parquet
ls step_2_highly_aggregated_notimputed.parquet
ls step_2_highly_classifications.csv
ls aggregated_groups_highly.csv
ls missing_values_report_highly.csv
ls variance_report_highly.csv
ls pipeline_summary_highly.csv

# Mildly aggregated branch
ls step_2_mildly_aggregated.parquet
ls step_2_mildly_aggregated_notimputed.parquet
ls step_2_mildly_classifications.csv
ls aggregated_groups_mildly.csv
ls missing_values_report_mildly.csv
ls variance_report_mildly.csv
ls pipeline_summary_mildly.csv
```

2. Verify dimensions:

Hide

```
import pandas as pd
df_ml = pd.read_parquet('NLSY79_aggregated.parquet')
assert df_ml.shape == (7060, 1424)
```

3. Check completeness:

Hide

Hide

```
assert df_ml.isnull().sum().sum() == 0
```

4. Validate scaling:

Hide

```
classifications = pd.read_csv('ml_ready_classifications.csv')
continuous_cols = classifications[
    classifications['variable_type'].isin(['continuous', 'categorical_ordinal', 'year'])
]['variable_name'].tolist()

import numpy as np
assert np.allclose(df_ml[continuous_cols].mean(), 0, atol=1e-6)
assert np.allclose(df_ml[continuous_cols].std(), 1, atol=1e-6)
```

Troubleshooting

Common Issues & Solutions

Issue 1: “Classified data not found”

Error Message:

```
FileNotFoundError: [Errno 2] No such file or directory: 'nlsy79_classified.parquet'
```

Cause: Step 1 has not been run yet

Solution:

Hide

```
# Run Step 1 first to generate classified data
python step_1_classification.py

# Then run Step 2
python step_2_aggregation.py
```

Verification:

Hide

```
import os
assert os.path.exists('nlsy79_classified.parquet'), "Run step_1_classification.py first"
```

Issue 2: High Memory Usage

Symptoms: - Script crashes with `MemoryError` - System freezes during aggregation - Slow performance (>30 minutes runtime)

Expected Requirements: - RAM: 8-16 GB for highly aggregated branch - RAM: 16-32 GB for mildly aggregated branch (larger feature set)

Solutions:

Option 1: Process in chunks

Hide

```
# Modify script to process participant subsets
chunk_size = 1000
for i in range(0, len(df), chunk_size):
    chunk = df.iloc[i:i+chunk_size]
    # Process chunk
```

Option 2: Reduce feature set - Increase minimum timepoint threshold (e.g., ≥5 instead of ≥3 for highly aggregated) - Focus on single branch instead of dual-branch processing

Option 3: Increase system memory - Close other applications - Use cloud computing (AWS, Azure, Google Cloud)

Issue 3: Inconsistent Variable Groups Not Handled

Symptoms: - Console shows `Found 58 inconsistent variable groups` - Variables with mixed types in same group - Aggregation uses default (majority) type

Explanation: Some variable groups contain mixed types (e.g., `INCOME_1990` continuous, `INCOME_2000` ordinal)

Solution: Manual Review (Optional)

1. Check for review file:

Hide

```
import os
if os.path.exists('inconsistent_variables_review.csv'):
    print("✓ Review file exists - fill in corrections")
else:
    print("✗ Review file not generated by Step 1")
```

2. Open `inconsistent_variables_review.csv` and fill in:
    - `corrected_type` : Override classification
    - `aggregate_with_group` : YES / NO / YES1 / YES2
    - `notes` : Document reasoning
3. Rerun Step 2 - corrections automatically applied

Alternative: Accept Default Behavior - Pipeline uses majority type (most common across group) - Generally works well for most groups - Manual review only needed for critical variables

---

Issue 4: Unexpected Dimensionality Reduction

Symptoms: - Final dataset smaller than expected - Many variables removed

Cause: Low-variance filter removed near-constant variables

Check Removed Variables:

Hide

```
variance_report = pd.read_csv('variance_report_highly.csv')

# Variables removed (≥95% identical)
removed = variance_report[variance_report['unique_pct'] < 5.0]
print(f"Removed {len(removed)} low-variance variables")
print(removed[['variable_name', 'unique_pct', 'most_common_pct']].head(10))
```

Understanding Variance Filter:

Hide

```
# Example: Variable with 95%+ identical values
example_var = df['rare_program_participation']
value_counts = example_var.value_counts()
most_common_pct = value_counts.iloc[0] / len(example_var) * 100

if most_common_pct >= 95:
    print(f"✗ Removed: {most_common_pct:.1f}% are value {value_counts.index[0]}")
else:
    print(f"✓ Kept: sufficient variance ({100-most_common_pct:.1f}% variation)")
```

Solution (if needed): - Adjust variance threshold in script (default: 95%) - Accept dimensionality reduction (uninformative variables removed)

Issue 5: Imputation Not Working Correctly

Symptoms: - Non-imputed version has 0 missing values (should have missing) - Imputed and non-imputed versions are identical

Verification:

Hide

```
df_imp = pd.read_parquet('step_2_highly_aggregated.parquet')
df_notimp = pd.read_parquet('step_2_highly_aggregated_notimputed.parquet')

print(f"Imputed missing: {df_imp.isnull().sum().sum()} (should be 0)")
print(f"Non-imputed missing: {df_notimp.isnull().sum().sum()} (should be >0)")
print(f"Datasets identical: {df_imp.equals(df_notimp)} (should be False)")
```

Expected Behavior: - Imputed version: 0 missing values - Non-imputed version: Missing values preserved (e.g., 1,027,241 for highly aggregated)

If Issue Persists: - Check if participant filter removed all high-missingness rows - Verify missing value standardization in Step 1 - Re-run both Step 1 and Step 2

Issue 6: Wrong Variable Types After Aggregation

Symptoms: - Mode features classified as continuous (should be categorical_nominal) - Binary proportions classified incorrectly

Cause: Post-aggregation reclassification logic error

Verification:

Hide

```
types = pd.read_csv('step_2_highly_classifications.csv')

# Check mode features
mode_features = types[types['variable_name'].str.contains('_mode_')]
print("Mode features:")
print(mode_features[['variable_name', 'variable_type']].head())
# Expected: all should be 'categorical_nominal'

# Check continuous features
mean_features = types[types['variable_name'].str.contains('_mean_')]
print("\nMean features:")
print(mean_features[['variable_name', 'variable_type']].head())
# Expected: all should be 'continuous'
```

Solution: Check reclassification logic in script (Stage 6)

Issue 7: Manual Corrections Not Applied

Symptoms: - Console doesn't show "Applied X type corrections" - Variables still use original (incorrect) classifications - Subgroups not created

Diagnostic:

Hide

```
import os
import pandas as pd

# Check if review file exists
if not os.path.exists('inconsistent_variables_review.csv'):
    print("✗ Review file not found - generate from Step 1")
else:
    review = pd.read_csv('inconsistent_variables_review.csv')

    # Check for filled-in corrections
    corrections = review[review['corrected_type'].notna() & (review['corrected_type'] != '')]
    subgroups = review[review['aggregate_with_group'].isin(['YES1', 'YES2', 'YES3'])]
    exclusions = review[review['aggregate_with_group'] == 'NO']

    print(f"✓ Type corrections: {len(corrections)}")
    print(f"✓ Subgroup assignments: {len(subgroups)}")
    print(f"✓ Exclusions: {len(exclusions)}")

    if len(corrections) == 0 and len(subgroups) == 0:
        print("\n⚠ No corrections found in review file")
        print("   Fill in 'corrected_type' or 'aggregate_with_group' columns")
```

Solution: 1. Ensure review file is in same directory as script 2. Verify columns are spelled correctly (`corrected_type` , `aggregate_with_group`) 3. Remove extra spaces from cell values 4. Rerun Step 2

Issue 8: Slow Aggregation Performance

Symptoms: - Script runs >30 minutes (expected: 3-5 minutes for highly, 5-10 minutes for mildly) - Hangs during aggregation stage

Optimization Tips:

Check System Resources:

Hide

```
import psutil
print(f"CPU usage: {psutil.cpu_percent()}%")
print(f"RAM usage: {psutil.virtual_memory().percent}%")
```

Reduce Computational Load: - Process single branch at a time - Increase minimum timepoint threshold (reduces groups to aggregate) - Use multi-core processing (modify script to use `joblib` or `multiprocessing`)

Hardware Recommendations: - CPU: 4+ cores (8+ recommended) - RAM: 16 GB minimum (32 GB recommended for mildly aggregated) - Storage: SSD for faster I/O

Issue 9: Output Files Missing or Incomplete

Expected Output Files per Branch: - `step_2_{branch}_aggregated.parquet` (imputed) - `step_2_{branch}_aggregated_notimputed.parquet` - `step_2_{branch}_classifications.csv` - `aggregated_groups_{branch}.csv` - `missing_values_report_{branch}.csv` - `variance_report_{branch}.csv` - `pipeline_summary_{branch}.csv` - `inconsistent_variables_report_{branch}.txt`

Diagnostic:

Hide

```
import os

branches = ['highly', 'mildly']
expected_files = [
    'step_2_{}_aggregated.parquet',
    'step_2_{}_aggregated_notimputed.parquet',
    'step_2_{}_classifications.csv',
    'aggregated_groups_{}.csv',
    'missing_values_report_{}.csv',
    'variance_report_{}.csv',
    'pipeline_summary_{}.csv'
]

for branch in branches:
    print(f"\n{branch.upper()} BRANCH:")
    for file_pattern in expected_files:
        file_name = file_pattern.format(branch)
        exists = os.path.exists(file_name)
        print(f"  {'✓' if exists else '✗'} {file_name}")
```

Solution: - Check console output for errors during save stage - Ensure write permissions in directory - Verify sufficient disk space - Rerun Step 2 if files are missing

Getting Help

Still Having Issues?

1. Check Console Output: Look for error messages and warnings
2. Verify Input Files: Ensure Step 1 outputs exist and are valid
3. Review Logs: Check `pipeline_summary_{branch}.csv` for processing statistics
4. Inspect Intermediate Data: Load parquet files at each stage to diagnose

Debugging Template:

Hide

```
import pandas as pd
import numpy as np

print("DEBUGGING STEP 2")
print("=" * 80)

# 1. Check input
df = pd.read_parquet('nlsy79_classified.parquet')
print(f"Input dimensions: {df.shape}")
print(f"Input missing: {df.isnull().sum().sum():,}")

# 2. Check after participant filter
participants_before = len(df)
participants_after = len(df[df.isnull().mean(axis=1) <= 0.25])
print(f"Participants: {participants_before} -> {participants_after}")

# 3. Check variable grouping
# ... (add relevant diagnostic code)

print("=" * 80)
```

References

Trajectory Analysis: - Nagin, D. S. (2005). Group-Based Modeling of Development. Harvard University Press. - Singer, J. D., & Willett, J. B. (2003). Applied Longitudinal Data Analysis. Oxford University Press.

Feature Engineering: - Kuhn, M., & Johnson, K. (2019). Feature Engineering and Selection. CRC Press. - Zheng, A., & Casari, A. (2018). Feature Engineering for Machine Learning. O'Reilly Media.

Missing Data: - Little, R. J., & Rubin, D. B. (2019). Statistical Analysis with Missing Data (3rd ed.). Wiley. - Van Buuren, S. (2018). Flexible Imputation of Missing Data (2nd ed.). CRC Press.

Software: - Scikit-learn: https://scikit-learn.org/stable/ (https://scikit-learn.org/stable/) - Pandas: https://pandas.pydata.org/docs/ (https://pandas.pydata.org/docs/) - NumPy: https://numpy.org/doc/ (https://numpy.org/doc/)

Document generated: December 17, 2025
NLSY79 aggregation & ML Preparation Pipeline - Step 2